

AI-Assisted Inference Engine Research Project

Project Objective

You are an AI Systems Researcher and Inference Engineer.

Your mission is to design, benchmark, optimize, and analyze a local transformer inference engine using an autoresearch methodology inspired by the Flash-MoE project.

The goal is not to train a new model.

The goal is to:

1. Build a reproducible baseline inference engine.
2. Run at least 40 benchmarked optimization experiments.
3. Discover which optimizations actually improve performance.
4. Document both successful and failed experiments.
5. Build an optimized inference engine.
6. Validate optimization transferability on a large-scale MoE model.
7. Build a real-time observability dashboard demonstrating the differences between baseline and optimized inference.

The final result should resemble an inference-engine research project rather than a standard software engineering project.

Development Strategy

Phase 1: Development Model

Begin development using:

- Qwen/Qwen2.5-0.5B-Instruct

Purpose:

- Fast experimentation
- Fast benchmark execution
- Lower hardware requirements
- Rapid iteration
- Easier debugging

All infrastructure must first be built and validated using this model.

This includes:

- Baseline inference engine
 - Benchmark harness
 - Metrics collection
 - Experiment framework
 - Logging system
 - Dashboard framework
-

Phase 2: Large-Scale Validation

After the optimization cycle is complete, evaluate optimization transferability on:

- Qwen3.5-397B-A17B

The purpose is to determine whether optimizations discovered on the small model transfer to a large-scale Mixture-of-Experts architecture.

Do not perform optimization development directly on the large model.

The large model should only be used after:

1. Baseline engine is complete.
 2. Benchmark harness is complete.
 3. 40+ experiments have been executed.
 4. The optimized engine has been selected.
-

Baseline Engine Requirements

Create a reproducible baseline implementation using Hugging Face Transformers.

Measure:

- Tokens generated
- Tokens per second
- Time to first token
- Total latency
- Peak memory usage
- CPU utilization
- GPU utilization
- Context length

Store all baseline benchmark results.

Benchmark Harness

Create:

benchmark_runner.py

The benchmark suite should include:

Factual Prompts

Knowledge and QA tasks.

Coding Prompts

Code generation and debugging tasks.

Long Reasoning Prompts

Multi-step reasoning tasks.

Summarization Prompts

Document summarization tasks.

Structured Output Prompts

JSON, YAML, and table generation.

All benchmark runs must be reproducible.

AI Autoresearch Methodology

Follow an autoresearch workflow inspired by Flash-MoE.

The objective is not to perform 40 random experiments.

The objective is to discover through measurement, profiling, and iteration which inference optimizations actually matter.

For every experiment:

1. Analyze benchmark results.

2. Identify the largest bottleneck.
3. Form a hypothesis.
4. Implement a targeted optimization.
5. Benchmark.
6. Compare against current best.
7. Document findings.
8. Use findings to guide the next experiment.

Experiments should build on previous discoveries.

Bottleneck-Driven Development

Before every experiment:

Profile the system.

Identify:

- Compute bottlenecks
- Memory bottlenecks
- KV cache bottlenecks
- Token generation bottlenecks
- Scheduling bottlenecks
- Model loading bottlenecks

Prioritize experiments that attack the largest bottleneck.

Trust Measurements Over Intuition

Never assume an optimization helps.

Every optimization must be validated with benchmark results.

Measured data takes priority over intuition.

Never fabricate:

- benchmark results
- profiling results
- experiment outcomes
- speedups
- latency improvements

All reported metrics must come from actual execution.

Optimization Categories

Runtime Optimizations

- `torch.inference_mode()`
- `torch.no_grad()`
- `torch.compile()`
- optimized generation loops
- tokenizer optimizations
- graph capture opportunities

Memory Optimizations

- KV cache improvements
- cache layouts
- memory reuse
- pinned memory
- memory mapping

Quantization

- FP16
- BF16
- INT8
- 4-bit
- mixed precision

Scheduling

- prompt batching
- dynamic batching
- continuous batching
- request scheduling

Decoding

- speculative decoding
- draft models
- alternative generation strategies
- early exit strategies

System Optimizations

- multiprocessing
 - threading
 - async execution
 - model loading improvements
-

Minimum Experiment Requirement

Run at least:

40 unique benchmarked experiments.

Every experiment must contain:

Experiment Number: Title: Hypothesis: Implementation: Files Modified: Benchmark Results: Performance Change: Decision: Lessons Learned:

Failure Documentation

Failed experiments are valuable.

Do not hide failures.

For every failed experiment document:

- Why it was attempted
- Why it failed
- Performance impact
- Lessons learned

Maintain:

failure_log.md

Experiment Retention Policy

Keep changes only if they:

- Improve throughput

- Reduce latency
- Reduce memory usage
- Preserve output quality

Otherwise revert the change.

The final optimized engine should contain only validated improvements.

Required Outputs

results/

- experiments.csv
- experiments.json
- leaderboard.csv
- benchmark_history.csv
- best_config.json

experiments/

- exp001.md
- exp002.md
- ...
- exp040.md

reports/

- final_report.md
-

What We Tried (And What Worked)

Generate a dedicated report section containing:

- Top 10 successful optimizations
 - Top 10 failed optimizations
 - Largest speedup
 - Largest latency reduction
 - Largest memory reduction
 - Most surprising finding
 - Highest ROI optimization
 - Optimization that looked promising but failed
 - Optimization that improved one metric while harming another
-

Final Engine Selection

After all experiments:

Create:

baseline_engine.py

optimized_engine.py

The optimized engine must be the highest-performing validated configuration.

Large Model Transfer Validation

After the optimization cycle:

Apply only the top-performing optimizations to:

- Qwen3.5-397B-A17B

Do NOT repeat all 40 experiments.

Instead:

1. Benchmark large-model baseline.
2. Benchmark large-model optimized.
3. Compare results.
4. Measure transferability.

Research Question:

"Can optimizations discovered on a small transformer transfer successfully to a large-scale Mixture-of-Experts model?"

Interactive Inference Dashboard

IMPORTANT:

The dashboard must be built AFTER the optimization cycle is complete.

The dashboard is a Phase 2 deliverable.

Launch command:

```
streamlit run dashboard.py
```

The dashboard should function as a professional inference-engine observability and research platform.

Live Inference Playground

Users must be able to:

- Enter prompts
- Select model
- Select engine
- Configure generation parameters
- Run inference directly

Controls:

- Prompt textbox
- System prompt textbox
- Max tokens
- Temperature
- Top-p
- Top-k
- Seed

Model Selector:

- Qwen2.5-0.5B-Instruct
- Qwen3.5-397B-A17B

Engine Selector:

- Baseline Engine
 - Optimized Engine
 - Compare Both
-

Real-Time Streaming Generation

Generated text must stream live.

Do not wait for completion.

Display:

- Generated text
- Tokens generated
- Elapsed time
- Current tokens/sec
- Time to first token

Update continuously while inference is running.

Real-Time Performance Monitoring

While generation is executing display:

- Current tokens/sec
- Rolling tokens/sec
- Current latency
- Time to first token
- Peak memory usage
- Current memory usage
- CPU utilization
- GPU utilization
- KV cache size
- Context length
- Active model
- Active engine

Metrics should update live.

The dashboard should behave like an inference profiler.

Baseline vs Optimized Comparison Mode

Users must be able to run:

- Baseline only
- Optimized only
- Both simultaneously

Display side-by-side:

Baseline Output

Optimized Output

Performance Metrics:

- Tokens/sec
- Time to first token
- Total latency
- Memory usage
- CPU utilization
- GPU utilization

Calculate:

$\text{speedup} = \text{optimized_tps} / \text{baseline_tps}$

Display:

- Absolute improvement
 - Percentage improvement
-

Live Visualization

Display continuously updating charts:

- Tokens/sec over time
- Memory usage over time
- CPU usage over time
- GPU usage over time

Allow users to observe inference behavior while generation is occurring.

Experiment Explorer

Users should be able to:

- Browse experiments
- Sort experiments
- Filter experiments
- View accepted optimizations
- View rejected optimizations

For each experiment display:

- Hypothesis
 - Category
 - Results
 - Decision
 - Lessons learned
-

Optimization Timeline

Display:

- Experiment Number vs Tokens/sec
- Experiment Number vs Latency
- Experiment Number vs Memory Usage
- Experiment Number vs Speedup

Allow users to visualize how the inference engine evolved.

Optimization Diff Viewer

Display:

Baseline Configuration

vs

Optimized Configuration

Show:

- Runtime changes
- Quantization changes
- Cache changes
- Scheduling changes
- System-level changes

Allow users to understand why the optimized engine performs better.

MoE Visualization

For Qwen3.5-397B-A17B:

If expert routing information is accessible:

Display:

- Expert utilization
- Expert activation frequencies
- Expert load distribution
- Tokens routed per expert
- Expert heatmaps

If unavailable:

Document the limitation and expose the maximum available execution information.

Session Logging

Every dashboard run should optionally record:

- Prompt
- Model
- Engine
- Output
- Metrics
- Timestamp

Store in:

dashboard_logs/

Final User Experience Goal

A user should be able to:

1. Open the dashboard.
2. Enter a prompt.
3. Select a model.
4. Select baseline or optimized inference.
5. Watch generation stream live.

6. Observe tokens/sec changing in real time.
7. Observe memory and hardware utilization.
8. Compare baseline and optimized performance.
9. Explore the optimization history.
10. Understand exactly how the optimized engine was created.

The dashboard should feel like a professional inference-engine research and observability platform rather than a simple web application.